

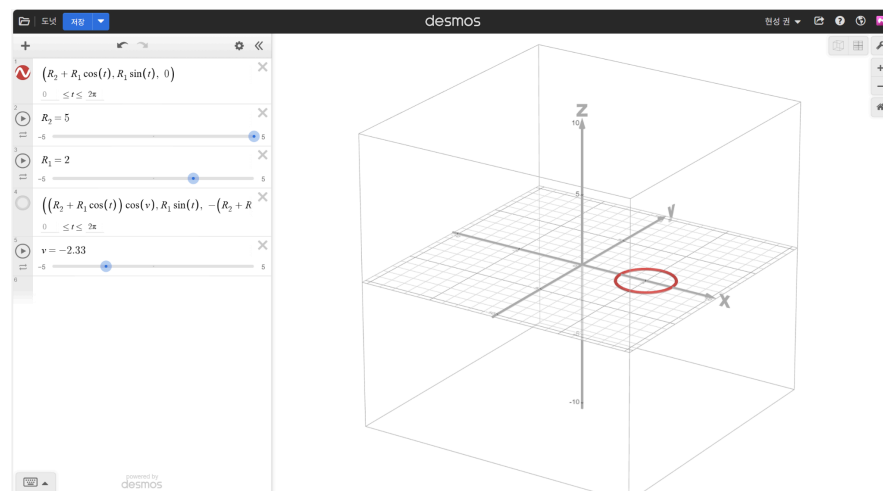
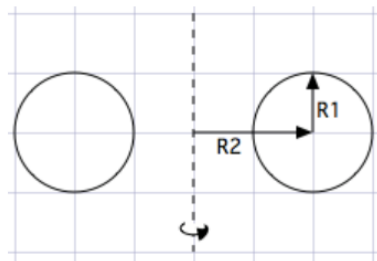
회전 도넛 만들기 - 30405 권현성

참고 - [I Coded a 3D Spinning Donut](#) , [How does the Spinning Cube work?](#) , [Donut math: how donut.c works](#), [ASCII-Donut-Animation/src/circle.cpp at main · sherwinvishesh/ASCII-Donut-Animation · GitHub](#)

원리 설명 : 우선 도넛을 만들어야 한다. (desmos 프로그램을 이용해서 확인)

1. 원점으로부터 일정거리 떨어진 원을 만든다. (빨간색 원)
 - 이 부분은 벡터의 합성과 같다.

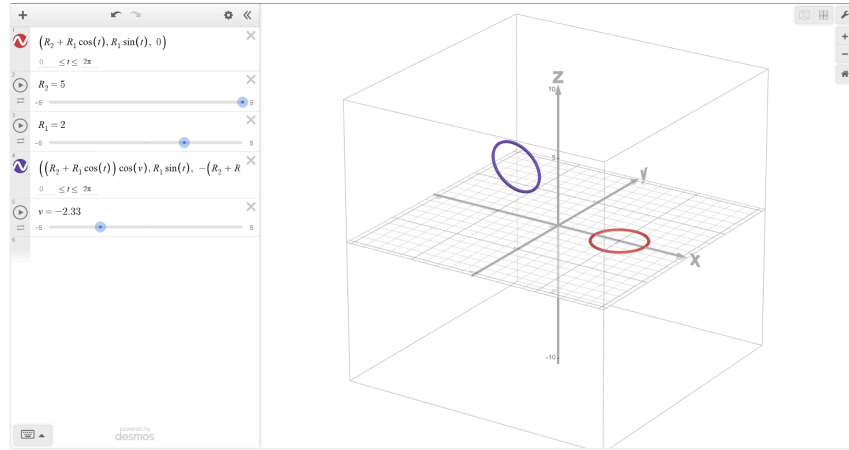
$$(x, y, z) = (R_2, 0, 0) + (R_1 \cos \theta, R_1 \sin \theta, 0)$$



2. 그 원을 y축기준으로 회전시킨다. 이를 반복문을 이용하면 도넛을 만들 수 있다. (보라색 원)
 - 앞에서 만든 원에 회�행렬을 점급하면 구할 수 있다. (보라색 원)

$$(R_2 + R_1 \cos \theta, R_1 \sin \theta, 0) \cdot \begin{pmatrix} \cos \phi & 0 & \sin \phi \\ 0 & 1 & 0 \\ -\sin \phi & 0 & \cos \phi \end{pmatrix}$$

$$= ((R_2 + R_1 \cos \theta) \cos \phi, R_1 \sin \theta, -(R_2 + R_1 \cos \theta) \sin \phi)$$



- 참고 : 회전형렬은 x, y, z 축 회전 행렬이 있다. 이는 각각의 축을 기준으로 회전 할 수 있음을 의미한다.

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

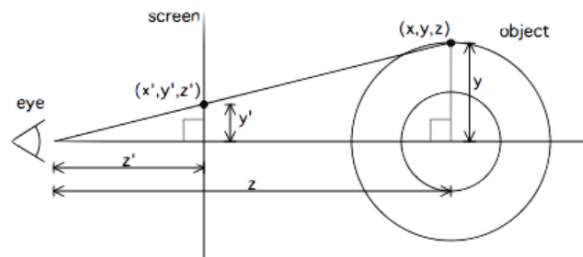
3. 도넛을 만들었다면 이를 회전시켜야 한다. 이는 아까처럼 회전형렬을 곱하면 되는데, x 축, z 축으로 회전을 시킬 것이기 때문에 x, z 축 행렬을 곱하면 된다.

$$(R_2 + R_1 \cos \theta, R_1 \sin \theta, 0) \cdot \begin{pmatrix} \cos \phi & 0 & \sin \phi \\ 0 & 1 & 0 \\ -\sin \phi & 0 & \cos \phi \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos A & \sin A \\ 0 & -\sin A & \cos A \end{pmatrix} \cdot \begin{pmatrix} \cos B & \sin B & 0 \\ -\sin B & \cos B & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- 아래는 계산을 했을 때의 식이다.

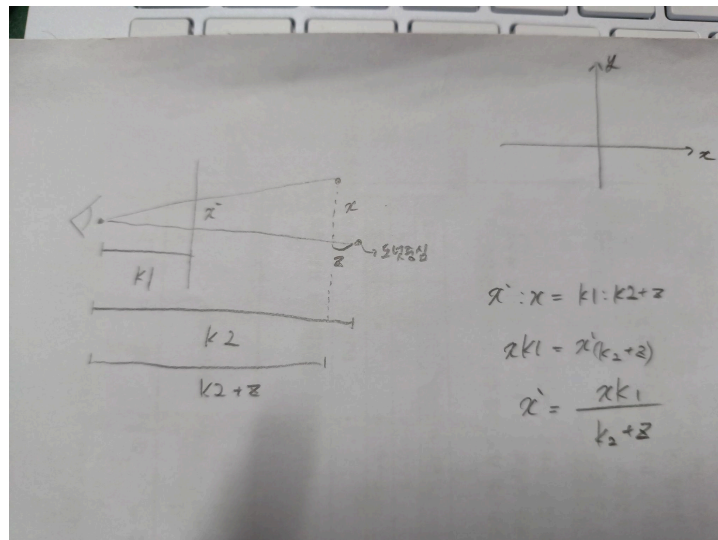
$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} (R_2 + R_1 \cos \theta)(\cos B \cos \phi + \sin A \sin B \sin \phi) - R_1 \cos A \sin B \sin \theta \\ (R_2 + R_1 \cos \theta)(\cos \phi \sin B - \cos B \sin A \sin \phi) + R_1 \cos A \cos B \sin \theta \\ \cos A(R_2 + R_1 \cos \theta) \sin \phi + R_1 \sin A \sin \theta \end{pmatrix}$$

4. 그다음은 3차원 도형을 만들었으니 2차원에 투영을 하는 방법이다. 직각삼각형의 닮음을 이용해서 화면에 투영을 할 수 있다.
- 2D평면에 투영을 할때 x, y 는 똑같은 방법으로 투영을 할 수 있다.



$$\frac{y'}{z'} = \frac{y}{z}$$

$$y' = \frac{yz'}{z}$$



- 그렇다면 깊이정보는 어떻게 하는가?, 뒤에 있는것과 앞에 있는 것을 구별해서 앞에 있는 것이 보이게 해야한다. 이를 위해서 **z-buffer**를 사용한다.
- 지금 위의 그림에서 화면에 투영해야하는 z (깊이)정보를 담는 것인데 $1/z$ (z 의 역수)를 사용한다. 이러면 무한히 멀면 0이되고, 가까울수록 값이 크다.

일단 여기까지 코딩을 해보자

그러기 위해서는 **ANSI Escape Sequence**와 터미널에 대해서 알아야한다. (검색과 gpt를 통해 알아보았다.)

- **ANSI Escape Sequence** : 터미널을 제어하기 위해서 사용되는 표준이라고 한다. 터미널에서 커서위치, 색, 글자 스타일 등을 제어할 수 있다.

예) 출처 : [C/C++ \] ANSI 이스케이프 코드 시퀀스 사용해서 커서 이동 + 터미널 조작하기](#)

커서 숨기기 : `\x1B[?25l`

전체 화면 지우기 : `\x1B[2J`

특정 위치로 커서 이동 : `\x1B[<row>;<col>H`, row는 행번호 col은 열번호로 `\x1B[0;0H` 또는 `\x1B[1;1H` 는 화면의 좌측 상단 첫번째 칸으로 이동한다.

텍스트 색상 변경 : `\x1B[38;5;<color>m`, 여기서 color는 색상 코드이다.

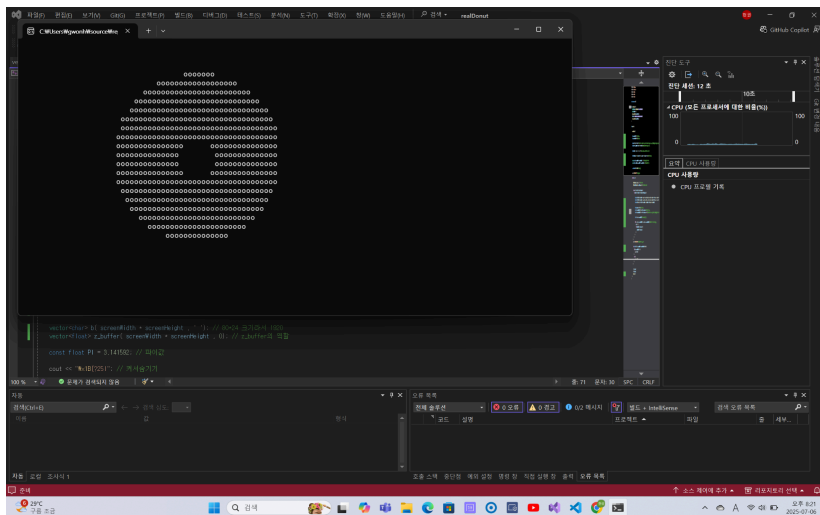
- 터미널 : 명령어를 입력하고 결과를 문자기반으로 보여주는 텍스트 인터페이스
- 근데 **windows** 콘솔은 **ANSI** 시퀀스를 기본적으로는 지원하지 않는다고 한다. 그래서 GPT에게 물어보았다. **Virtual Terminal Processing**을 켜면 가능하다고 한다.

```
#include <windows.h>
```

```
void enableAnsi() {
    HANDLE hOut = GetStdHandle(STD_OUTPUT_HANDLE);
    DWORD dwMode = 0;
    GetConsoleMode(hOut, &dwMode);
    dwMode |= ENABLE_VIRTUAL_TERMINAL_PROCESSING;
    SetConsoleMode(hOut, dwMode);
}
```

- 위의 **enableAnsi()** 함수만 호출하면 사용 가능하다고 한다.
- 그리고 터미널은 문자셀 단위로 동작한다. 나는 가로 **80**,세로 **24**로 화면 출력 크기를 정했다.

중간 코딩결과 - 어찌저찌 도넛의 형상을 구현하는데에 성공했다. (다른 사람들의 소스코드를 보면서 했는데도 쉽지 않았다.) *근데 밝기와 색을 구현하고 알았는데 **z**좌표를 구현하는데에 **sin**값을 잘못넣었다는 것을 한참을 보고서 알게 되었다. 그래서 이 도넛은 형태가 이상하다.



```
#include <iostream>
#include <windows.h>
#include <string>
#include <cmath>
#include <vector>
```

```
using namespace std;
```

```
void enableAnsi() {
```

```

HANDLE hOut = GetStdHandle(STD_OUTPUT_HANDLE);
DWORD dwMode = 0;
GetConsoleMode(hOut, &dwMode);
dwMode |= ENABLE_VIRTUAL_TERMINAL_PROCESSING;
SetConsoleMode(hOut, dwMode);
}

int main() {

    enableAnsi();

    int screenWidth = 80; //화면가로
    int screenHeight = 24; //화면세로

    float R1 = 1, R2 = 2, K2 = 5; // R1: 원 중심에서 반지름, R2: 원점에서 원중심까지 거리,
    K2: 도넛중심에서 시점까지의 거리
    float K1 = screenWidth * K2 / (5 * (R1 + R2)); // K1: 화면에서 시점까지의 거리

    float theta = 0, pi = 0, A = 0, B = 0; //각들_ theta, pi: 도넛 / A,B : 회전

    float theta_plus = 0.06, pi_plus = 0.02, A_plus = 0.02, B_plus = 0.04; //각들 증가값

    vector<char> b( screenWidth * screenHeight , ' '); // 80*24 크기라서 1920
    vector<float> z_buffer( screenWidth * screenHeight , 0); // z_buffer의 역할

    const float PI = 3.141592; // 파이값

    cout << "\x1B[?25I"; // 커서숨기기

    while (true) {

        fill(b.begin(), b.end(), ' '); // b 초기화
        fill(z_buffer.begin(), z_buffer.end(), 0); // z_buffer 초기화

        for (pi = 0; pi < PI * 2; pi += pi_plus) {
            for (theta = 0; theta < PI * 2; theta += theta_plus) {

                float x = (R2 + R1 * cos(theta)) * (cos(B) * cos(pi) + sin(A) * sin(B) * sin(pi)) - R1
                * cos(A) * sin(B) * sin(theta);
                float y = (R2 + R1 * cos(theta)) * (cos(pi) * sin(B) - cos(B) * sin(A) * sin(pi)) + R1
                * cos(A) * cos(B) * sin(theta);
                float z = K2 + cos(A) * (R2 + R1 * cos(theta)) * sin(theta) + R1 * sin(A) *
                sin(theta);

                float z_inverse = 1 / z; // z 역수 값
                int xp = screenWidth / 2 + (x * K1 * z_inverse); // 화면에서 x좌표
                int yp = screenHeight / 2 - (y * K1 * z_inverse * 0.5); // 화면에서 y좌표 ( y값은
                위로 갈수록 커지지만, 배열은 아래로 갈수록 커짐 )

                int n = xp + yp * screenWidth; // vector인덱스 값

                if (0 < xp && xp < screenWidth && 0 < yp && yp < screenHeight) { //화면의 범위
                안 인가를 판별
                    b[n] = 'o';

```

```

        if (z_buffer[n] < z_inverse) {
            z_buffer[n] = z_inverse;
        }
    }
}
}
cout << "\x1B[0;0H"; // 커서 제일 위로 옮기기

for (int i = 0; i < screenWidth * screenHeight + 1; i++) {
    if (i % screenWidth != 0) {
        cout << b[i];
    }
    else {
        cout << endl;
    }
}

A += A_plus;
B += B_plus;
Sleep(10);
}
}

```

이제는 **밝기**를 구현해야한다. 밝기는 문자로 구별한다. 가장밝은 쪽은 “@”, 가장어두운 쪽은 “.” 이런식이다. 이번에도 회전 행렬을 이용해서 밝기를 추측한다고 한다(이거는 이해못했다).

So our surface normal (N_x, N_y, N_z) is derived the same as above, except the point we start with is just $(\cos \theta, \sin \theta, 0)$. Then we apply the same rotations:

$$(N_x, N_y, N_z) = (\cos \theta, \sin \theta, 0) \cdot \begin{pmatrix} \cos \phi & 0 & \sin \phi \\ 0 & 1 & 0 \\ -\sin \phi & 0 & \cos \phi \end{pmatrix} \cdot \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos A & \sin A \\ 0 & -\sin A & \cos A \end{pmatrix} \cdot \begin{pmatrix} \cos B & \sin B & 0 \\ -\sin B & \cos B & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

So which lighting direction should we choose? How about we light up surfaces facing behind and above the viewer: $(0, 1, -1)$. Technically this should be a normalized unit vector, and this vector has a magnitude of $\sqrt{2}$. That's okay -- we will compensate later. Therefore we compute the above (x,y,z) , throw away the x and get our luminance $L = y-z$.

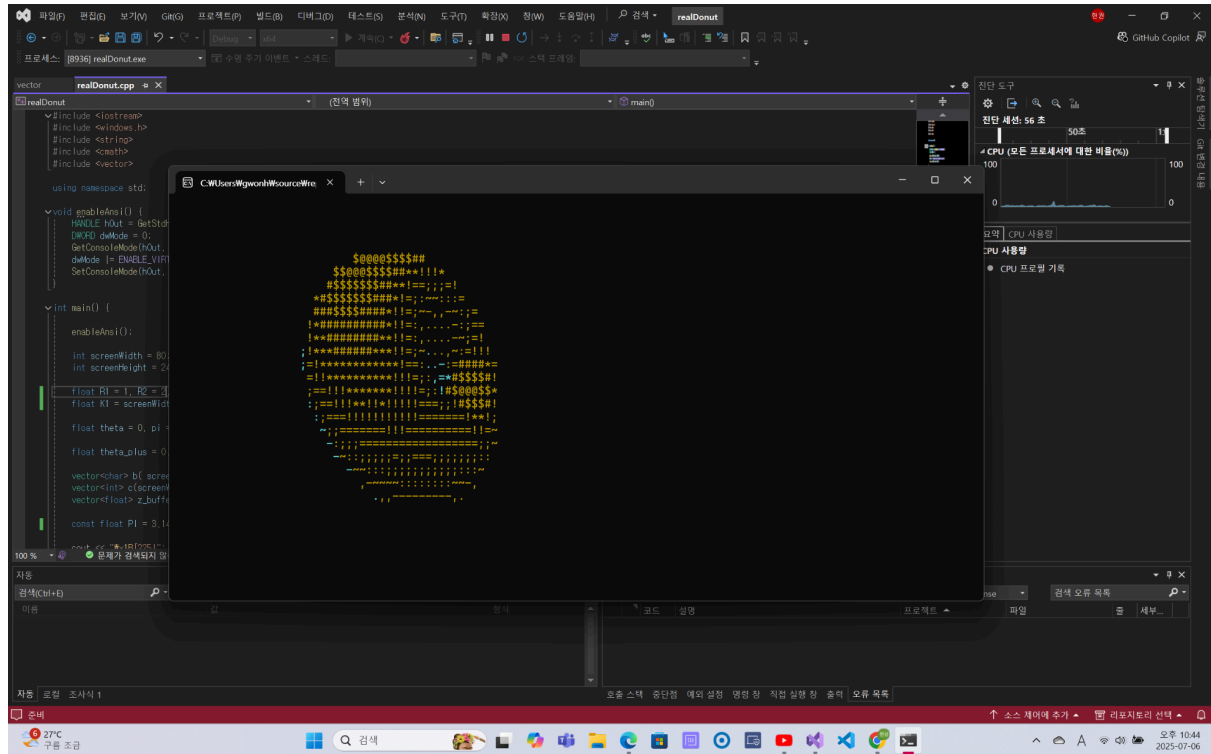
$$L = (N_x, N_y, N_z) \cdot (0, 1, -1) \\ = \cos \phi \cos \theta \sin B - \cos A \cos \theta \sin \phi - \sin A \sin \theta + \cos B (\cos A \sin \theta - \cos \theta \sin A \sin \phi)$$

- 아무튼 코드는 위의 식의 값이 0보다 클때, 8을 곱해준다음 .,~::~;=!*#\$@ 이 순서로 적용한다.
- 0보다 작을때는 그냥 0으로 만들어 줬다.

그리고 **색**도 넣어줬다.

- 색은 **ASNI** 이스케이프 문자를 이용해서 넣어주었다.

최종결과물과 코드



```
#include <iostream>
#include <windows.h>
#include <string>
#include <cmath>
#include <vector>
```

```
using namespace std;
```

```
void enableAnsi() {
    HANDLE hOut = GetStdHandle(STD_OUTPUT_HANDLE);
    DWORD dwMode = 0;
    GetConsoleMode(hOut, &dwMode);
    dwMode |= ENABLE_VIRTUAL_TERMINAL_PROCESSING;
    SetConsoleMode(hOut, dwMode);
}
```

```
int main() {

    enableAnsi();

    int screenWidth = 80; //화면가로
    int screenHeight = 24; //화면세로
```

```

float R1 = 1, R2 = 2, K2 = 5; // R1: 원 중심에서 반지름, R2: 원점에서 원중심까지 거리,
K2: 도넛중심에서 시점까지의 거리
float K1 = screenWidth * K2 / (5*(R1 + R2)); // K1: 화면에서 시점까지의 거리

float theta = 0, pi = 0, A = 0, B = 0; //각들_theta, pi: 도넛 / A,B : 회전

float theta_plus = 0.06, pi_plus = 0.02, A_plus = 0.04, B_plus = 0.02; //각들 증가값

vector<char> b( screenWidth * screenHeight , ' '); // 80*24 크기라서 1920
vector<int> c(screenWidth * screenHeight, 0); // 색상
vector<float> z_buffer( screenWidth * screenHeight , 0); // z_buffer의 역할

const float PI = 3.141592; // 파이값

cout << "\x1B[?25l"; // 커서숨기기
cout << "\x1b[2J"; // 전체화면 지우기
cout << "\x1b[0m"; // 기본색상으로 돌아감

while (true) {

    fill(b.begin(), b.end(), ' '); // b 초기화
    fill(z_buffer.begin(), z_buffer.end(), 0); // z_buffer 초기화

    for (theta = 0; theta < PI * 2; theta = theta + theta_plus) {
        for (pi = 0; pi < PI * 2; pi = pi + pi_plus) {

            float x = (R2 + R1 * cos(theta)) * (cos(B) * cos(pi) + sin(A) * sin(B) * sin(pi)) - R1
            * cos(A) * sin(B) * sin(theta); // x좌표
            float y = (R2 + R1 * cos(theta)) * (cos(pi) * sin(B) - cos(B) * sin(A) * sin(pi)) + R1
            * cos(A) * cos(B) * sin(theta); // y좌표
            float z = K2 + cos(A) * (R2 + R1 * cos(theta)) * sin(pi) + R1 * sin(A) * sin(theta);
            // z좌표 - sin(pi)를 sin(theta)로 썼네 이거 때문에 계속 들여다 봤네 ㅠㅠ

            float z_inverse = 1 / z; // z 역수 값
            int xp = screenWidth / 2 + (x * K1 * z_inverse); // 화면에서 x좌표
            int yp = screenHeight / 2 - (y * K1 * z_inverse * 0.5); // 화면에서 y좌표 ( y값은
            위로 갈수록 커지지만, 배열은 아래로 갈수록 커짐 )

            // 빛
            float luminance = cos(pi) * cos(theta) * sin(B) - cos(A) * cos(theta) * sin(pi) -
            sin(A) * sin(theta) + cos(B) * (cos(A) * sin(theta) - cos(theta) * sin(A) * sin(pi));

            int n = xp + yp * screenWidth;

            if (luminance < 0) { // 중간에 도넛이 떨어져서 그냥 음수는 0으로하기로 했다
                luminance = 0;
            }

            if (xp > 0 && xp < screenWidth && yp > 0 && yp < screenHeight) {
                if (z_buffer[n] < z_inverse) {
                    int luminance_index = 8 * luminance;
                    z_buffer[n] = z_inverse;
                    b[n] = ".,-~:;=!*$@"[luminance_index];
                    c[n] = (theta < PI) ? 1 : 0;
                }
            }
        }
    }
}

```



```

    }
    }
}
cout << "\x1B[0;0H"; // 커서 제일 위로 옮기기

for (int i = 0; i < screenWidth * screenHeight; i++) {
    if (c[i] == 0) {
        cout << "\033[96m"; // 밝은 하늘색
    }
    else {
        cout << "\033[33m"; // 노란색
    }

    if (i % screenWidth != 0) {
        cout << b[i]; // 출력
    }
    else {
        cout << endl; // 줄바꿈
    }
}

A += A_plus;
B += B_plus;
Sleep(20); // 딜레이
}
}

```

느낀점

- 처음에는 그저 영상을 보고 재미있겠다라고 생각해서 시작했는데 막상 시작하고 나니 쉽지않았다.
- 도넛모양을 만들때 원을 그리는 방법이 기하에서 배운 벡터와 유사해서 쉽게 이해할 수 있었다. 그리고 행렬연산을 이용해서 좌표상의 점을 이동시키거나 변환할 수 있다는 것은 알고 있었지만 회전 행렬을 이용해서 직접 회전을 구현해보니까 더 잘 이해할 수 있게 되었다.
- 그리고 3D입체를 2D평면에 구현 할때 **z-buffer**와 삼각형의 닳음비를 활용하였는데 이를 통해서 수학적인 방법을 코딩으로써 구현하는데에 좀 더 익숙해지고 자신감이 생긴것 같다.
- 터미널에서 문자를 원하는 방식으로 출력하기 위해서 **ANSI 이스케이프 시퀀스**가 무엇인지 찾아보고 알아보았으며, 원하는 곳으로 커서를 이동하고, 커서를 숨기고, 색을 바꾸는 등 적절히 적용하여서, 프로그램을 구현할 수 있었다.
- 사실은 코딩을 하는 순수한 실력은 뛰어나지 않기 때문에, 유튜브도 열심히 찾아보고 다른 사람들의 소스코드도 열심히 찾아보았다. 사실 보고 안 되면 따라한게 다지만 버그가 많이나고 프로그램이 제대로 굴러가지 않아서 힘들었다.
- **ai**가 도움이 되는 것도 있지만 너무 답답한 부분이 많았다. **ai**도 유능하지만 내가 직접 하는게 꼭 필요하다고 느꼈다.

- 그리고 도넛이 모양이 너무 이상해서 뭐가 잘못된건지 한참을 찾았는데, 중간에 정말 **sin값** 하나를 잘못넣은 것을 발견했을때는 안도감과 분노를 둘다 느꼈다. 그리고 고친 다음에 정상적인 도넛이 되었을때는 정말 속이 시원했다. 이걸로 느낀건 잘못된게 생겼을때는 정말 차분하게 처음부터 꼼꼼히 돌아봐야한다는 생각이 들었다. 그리고 잘못된 코드를 찾는 과정에서 인내심을 기를 수 있었다.